

Setuid Privileged Programs

Amir Masoumzadeh

CSI 424/524

Sep. 10, 2026



UNIVERSITY
AT ALBANY
State University of New York

Based on slides by Wenliang Du

Need for Privileged Programs

- Passwords are stored in `/etc/shadow`

```
1 $ sudo cat /etc/shadow
2 root:REDACTED:18315::::::
3 bin:!!:18315::::::
4 daemon:!!:18315::::::
5 mail:!!:18315::::::
6 systemd-resolve:!!:18315::::::
7 systemd-timesync:!!:18315::::::
8 systemd-coredump:!!:18315::::::
9 amir:REDACTED:18315:0:99999:7::
10 git:!!:18315::::::
11 cups:!!:18315::::::
```

- Writable only by its owner (root)

```
1 $ ls -l /etc/shadow
2 -rw----- 1 root root 1236 Aug  9 16:16 /etc/shadow
```

- How can regular users change their passwords?

Two-Tier Approach

- Implementing fine-grained access control in operating systems make OS overcomplicated
- OS relies on extensions to enforce fine-grained access control
- Privileged programs are such extensions

Types of Privileged Programs

- Daemons
 - Computer program that runs in the background
 - Needs to run as root or other privileged users
- setuid Programs
 - Widely used in UNIX systems
 - Program marked with a special bit

Setuid Concept

- Allow user to run a program with the **program owner's privilege**
- Privileges are temporarily elevated in the context of the program
- Example: the `passwd` program

```
1 $ ls -l /usr/bin/passwd
2 -rwsr-xr-x 1 root root 80856 Jun 27 03:35 /usr/bin/passwd
```

Setuid Concept

- Every process has two User IDs
 - Real UID (RUID) Identifies real owner of process
 - Effective UID (EUID) Used to determine privileges of process
- Access control is based on EUID
- When a normal program is executed: **RUID = EUID**
 - Both equal to ID of user who runs the program
- When a setuid program is executed: **RUID $\neq$ EUID**
 - RUID still equal to user's ID, but EUID equals to program owner's ID
 - If the program is owned by root, the program runs with root privilege

Turn a Program into setuid

- A setuid program is just like any other program, except that it has a special marking, a single bit called setuid bit

```
1 # Change the owner of a program to root:
2 $ cp /bin/cat ./mycat
3 $ sudo chown root mycat
4 $ ls -l mycat
5 -rwxr-xr-x 1 root seed 46764 Nov  1 13:09 mycat
6
7 # Before enabling setuid bit:
8 $ mycat /etc/shadow
9 mycat: /etc/shadow: Permission denied
10
11 # After enabling the setuid bit:
12 $ sudo chmod 4755 mycat
13 $ mycat /etc/shadow
14 root:$6$012BPz.KS$fPbKT6H6Db4/B8cLWBQl1cFjn/h/pDyc5U1BW0zkWh7T9ZGu.:15933::0:99999
```

How it Works

- We can check the RUID and EUID of a setuid program
- Program `id` is a good choice as it prints them for us

```
1 $ cp /bin/id ./myid
2 $ sudo chown root myid
3 $ ./myid
4 uid=1000(seed) gid=1000(seed) groups=1000(seed), ...
5
6 $ sudo chmod 4755 myid
7 $ ./myid
8 uid=1000(seed) gid=1000(seed) euid=0(root) ...
```

How is setuid Secure?

- Allows normal users to escalate privileges
 - This is different from directly giving the privilege (`sudo` command)
 - When using sudo we run a program using the real user ID of another user
 - Restricted behavior
- Unsafe to turn all programs into setuid
 - Example: `/bin/sh`
 - Example: `vi`

Attack Surfaces of setuid Programs

- ① User inputs
- ② System inputs that can be controlled by user
- ③ Environment variables
- ④ Capability leaking
- ⑤ Invoking non-privileged process controlled by user

Attacks via User Inputs

- Relying on explicit user inputs which could be maliciously crafted
- Example attacks that exploit them: (covered in future lectures)
 - Buffer Overflow: Overflowing a buffer to run malicious code
 - Format String Vulnerability: Changing program behavior using user inputs as format strings

Attacks via User Inputs (cont.)

- CHSH – Change Shell
 - setuid program with ability to change default shell programs
 - Shell programs are stored in `/etc/passwd` file
 - Potential vulnerability
 - Failing to sanitize user inputs
 - Attackers could create a new root account

Attacks via System Inputs

- Race Condition
 - Symbolic link to privileged file from an unprivileged file
 - Influence programs
 - Writing inside world-writable folder

Attacks via Environment Variables

- Behavior can be influenced by inputs that are not visible inside a program
- Environment Variables: These can be set by a user before running a program
- Example: `PATH` environment variable
 - Used by shell programs to locate a command if the user does not provide the full path for the command
 - Example: `system('ls')`
 - `system()` system call: calls `/bin/sh` first
 - `/bin/sh` uses the `PATH` environment variable to locate the `ls` command
 - Attacker can manipulate the `PATH` variable and control how the `ls` command is found (i.e., victim unknowingly runs a malicious version)

Capability Leaking

- In some cases, privileged programs downgrade themselves during execution
- Example: the `su` program
 - This is a privileged setuid program
 - Allows one user to switch to another user (e.g., user1 to user2)
 - Program starts with EUID as root and RUID as user1
 - After password verification, both EUID and RUID become user2's (via privilege downgrading)
- Such programs may forget to clean up privileged capabilities before downgrading, resulting in **capability leaking**

Attacks via Capability Leaking: An Example

```
1 fd = open("/etc/zzz", O_RDWR | O_APPEND);
2 if (fd == -1) {
3     printf("Cannot open /etc/zzz\n");
4     exit(0);
5 }
6
7 // Print out the file descriptor value
8 printf("fd is %d\n", fd);
9
10 // Permanently disable the privilege by making
11    ↪ the effective uid the same as the real uid
12 setuid(getuid());
13
14 // Execute /bin/sh
15 v[0] = "/bin/sh"; v[1] = 0;
16 execve(v[0], v, 0);
```

File descriptor is created by the setuid root program. File `/etc/zzz` is only writable by root

Downgrade privileges

Invoking a shell program lifts the expected behavior restriction

Attacks via Capability Leaking (Continued)

```
1 $ gcc -o cap_leak cap_leak.c
2 $ sudo chown root cap_leak
3 [sudo] password for seed:
4 $ sudo chmod 4755 cap_leak
5 $ ls -l cap_leak
6 -rwsr-xr-x 1 root seed 7386 Feb 23 09:24 cap_leak
7 $ cat /etc/zzz
8 bbbbbbbbbbbbbbbbbb
9 $ echo aaaaaaaaaa > /etc/zzz
10 bash: /etc/zzz: Permission denied
11 $ ./cap_leak
12 fd is 3
13 $ echo cccccccccc >&3
14 $ exit
15 $ cat /etc/zzz
16 bbbbbbbbbbbbbbbbbb
17 cccccccccc
```

Cannot write to the file

Using the leaked capability

File modified

- Program forgets to close the file, so file descriptor is still valid
- How to fix the program?
 - Destroy the file descriptor before downgrading the privilege (close the file)

Capability Leaking in OS X: Case Study

- OS X Yosemite found vulnerable to privilege escalation attack related to capability leaking in July 2015 (OS X 10.10)
- Added features to dynamic linker `dyld`
 - `DYLD_PRINT_TO_FILE` environment variable
- Dynamic linker can open any file, so for setuid root programs it runs with root privileges
- The dynamic linker `dyld` does not close the file, i.e., capability leaking
- Scenario 1 (safe): setuid program finishes its job and the process dies. Everything is cleaned up and it is safe
- **Scenario 2 (unsafe):** Similar to the `su` program, the privileged program downgrades its privilege, and lifts the restriction

Invoking Programs

- Invoking external commands from inside a program
- External command is chosen by the setuid program
 - Users are not supposed to provide the command (or it is not secure)
- Attack:
 - Users are often asked to provide input data to the command
 - If the command is not invoked properly, user's input data may be turned into command name

Invoking Programs: Unsafe Approach

- Easiest way to invoke an external command is using `system()`
- This program is supposed to run `/bin/cat`
- It is a setuid root program, so it can view all files, but it cannot write to any file
- Can you use this program to run another command, with the **root privilege**?

```
1 int main(int argc, char *argv[])
2 {
3     char *cat="/bin/cat";
4
5     if(argc < 2) {
6         printf("Please type a file name.\n");
7         return 1;
8     }
9
10    char *command = malloc(strlen(cat) +
11    ↪  strlen(argv[1]) + 2);
12    sprintf(command, "%s %s", cat, argv[1]);
13    system(command);
14    return 0;
15 }
```

Invoking Programs: Unsafe Approach (Continued)

```
1 $ gcc -o catall catall.c
2 $ sudo chown root catall
3 $ sudo chmod 4755 catall
4 $ ls -l catall
5 -rwsr-xr-x 1 root seed 7275 Feb 23 09:41 catall
6 $ ./catall /etc/shadow
7 root:*:18683:0:99999:7:::
8 daemon:*:18742:0:99999:7:::
9 bin:*:18742:0:99999:7:::
10 sys:*:18742:0:99999:7:::
11 sync:*:18742:0:99999:7:::
12 games:*:18742:0:99999:7:::
13 $ ./catall "aa;/bin/sh"
14 /bin/cat: aa: No such file or directory
15 #
16 # id
17 uid=1000(seed) gid=1000(seed) euid=0(root) groups=0(root),1000(seed)
```

We can get a root shell with this input

Got the root shell!

- Problem: Some part of the data becomes code (command name)

Note on Countermeasure in Ubuntu

- In Ubuntu, `/bin/sh` points to `/bin/dash`, which has a countermeasure
 - It drops privilege when it is executed inside a setuid process
 - Therefore, we will only get a normal shell in the attack on the previous slide
- Do the following to remove the countermeasure

```
1 $ sudo ln -sf /bin/zsh /bin/sh
```
- Remember to change it back after experimenting

```
1 $ sudo ln -sf /bin/dash /bin/sh
```

Invoking Programs Safely: Using `execve()`

```
1 int main(int argc, char *argv[])
2 {
3     char *v[3];
4
5     if(argc < 2) {
6         printf("Please type a file name.\n");
7         return 1;
8     }
9
10    v[0] = "/bin/cat"; v[1] = argv[1]; v[2] = 0;
11    execve(v[0], v, 0);
12
13    return 0;
14 }
```

Command name is provided here (by the program)

Input data is provided here (can be by the user)

- Why is it safe?
 - Code (command name) and data are clearly separated; there is no way for the user data to become code

Invoking Programs Safely (Continued)

```
1 $ gcc -o safecatall safecatall.c
2 $ sudo chown root safecatall
3 $ sudo chmod 4755 safecatall
4 $ ./safecatall /etc/shadow
5 root*:18683:0:99999:7::
6 daemon*:18742:0:99999:7::
7 bin*:18742:0:99999:7::
8 sys*:18742:0:99999:7::
9 sync*:18742:0:99999:7::
10 games*:18742:0:99999:7::
11 $ ./safecatall "aa;/bin/sh"
12 /bin/cat: aa;/bin/sh: No such file or directory
```

Attack failed!

Data is treated as data, not code

Additional Consideration

- Some functions in the exec family behave similarly to `execve()`, but may not be safe
 - `execlp()`, `execvp()` and `execvpe()` duplicate the actions of the shell, and can be attacked using the `PATH` environment variable

Invoking External Commands in Other Languages

- Risk of invoking external commands is not limited to C programs
- We should avoid problems similar to those caused by the `system()` function
- Examples:
 - Perl: the `open()` function can run commands, but it does so through a shell
 - PHP: the `system()` function

```
1 <?php
2 print ("Please specify the path of the directory");
3 print ("<p>");
4 $dir=$_GET['dir'];
5 print ("Directory path: ". $dir . "<p>");
6 system("/bin/ls $dir");
7 ?>
```

- `http://localhost/list.php?dir=.;date`
- Command executed on server: `/bin/ls .;date`

Principle of Isolation (Least Common Mechanism)

- Principle: **Don't mix code and data**
- Attacks due to violation of this principle:
 - `system()` code execution
 - Buffer overflow
 - Cross site scripting
 - SQL injection

Principle of Least Privilege

- A privileged program should be given the power which is required to perform its tasks
- Disable the privileges (temporarily or permanently) when a privileged program does not need those
- In Linux, `seteuid()` and `setuid()` can be used to disable/discard privileges
- Different OSes have different ways to achieve that